TAU - Advanced Topics in Programming, Semester B 2025

Assignment 3 (Link to Assignment 1, Link to Assignment 2)

Note: This document is in DRAFT mode till July-26th 23:00. While in draft mode, it may have changes without highlighting them. Even though it is still a draft, you may (and should) start working on it!

Changes after the draft period would be highlighted in the doc.
Note: if a change creates more work for you (e.g. you already implemented things differently), please raise the issue in the assignment forum before rushing to change your code!

Submission deadline: Sep 6th, 2026, 23:30.
Note: Submission Instructions appear inside the doc, below!

Assignment 3 - Requirements and Guidelines

In this exercise you shall add a **simulator** that can run **several drone algorithms** concurrently, using multithreading – in two modes: comparative or competitive.

Your exercise shall be separated into three parts, each one of them built separately with its own makefile, the parts are:

1. The MissionControl (MissionControl folder) - for running a single drone mission
2. The algorithmic part (Algorithm folders) - representing drone algorithm
3. The Simulator (Simulator folder) - running many drone missions in parallel

Each part above may run independently with another team's implementation of the other parts. Parts [1] and [2] would be compiled as shared libraries (.so) that would be dynamically loaded by the Simulation project.

You are required to organize your submission into **5 folders**: 3 folders for 3 separated projects as listed above, with a makefile inside each folder (1-3) and two common folders (4a and 4b), all are listed below:

1. **"Simulator"** folder: includes your Simulator class as well as other helper classes required by the simulator project. The executable name shall be: `simulator_<submitter_ids>` (for example: `simulator_098765432_123456789`)
2. **"Algorithm"** folder: in order to allow us to load different Algorithm ".so" files together, the name of the .so file must be unique, using the ids of the submitters, as follows: `Algorithm_098765432_123456789.so` additionally, all your code shall be located inside a namespace with the unique name `algorithm_098765432_123456789`
3. **"MissionControl"** folder: in order to allow load of different MissionControl ".so" files (together or separately), the name of the .so file must be unique, using the ids of the submitters, as follows: `MissionControl_098765432_123456789.so` additionally, all your code shall be located inside a namespace with the unique name `mission_control_098765432_123456789`
4. "common" and "UserCommon" (there are no makefiles in these folders!):

a. **“common”** folder: shall include the common files required by more than one project, published by the course staff as the files that shall be used “as is” without any change. DO NOT add your own files into this folder!
b. **“UserCommon”** folder: shall include your own common files, files that are required by more than one project. Your makefiles and includes may use those files where needed to avoid duplication. All your code in this folder shall be located inside a namespace with the unique name `user_common_098765432_123456789`

The Simulator

The Simulator is able to run several MissionControl objects (of the same MissionControl class type, or different class types).
Algorithms and the MissionControl(s) are loaded dynamically as .so files, as explained below.
The Simulator has two modes of operation:

1. Comparative run:
```
./simulator_<submitter_ids> -comparative
simulation=<simulation_composition_yaml_filename>
mission_control_folder=<mission_control_folder>
algorithm=<algorithm_so_filename> [num_threads=<num>] [-verbose]
```
2. Competition run:
```
./simulator_<submitter_ids> -competition
simulation=<simulation_composition_yaml_filename>
mission_control=<mission_control_so_filename>
algorithms_folder=<algorithms_folder> [num_threads=<num>] [-verbose]
```

Command line arguments for the two modes:

- All command line arguments can appear in any order.
- All command line arguments are mandatory.
- You may assume that the ‘=’ sign should appear without spaces around (as in: `simulation=<simulation_composition_yaml_filename>`, etc.).
- If unsupported command lines arguments are provided, the program should print a usage with an error message pointing at all the unsupported command lines arguments provided, then finish.
- In case command line arguments are missing, the program should print a usage with an error message detailing the missing command lines arguments, then finish.
- In case a command line argument that should point at a file is pointing at a non-existing file or one that cannot be opened, the program should print a usage with a proper error message, then finish.
- In case a command line argument that should point at a folder is pointing at a non-existing folder or one that cannot be traversed or at a folder that has zero files of the desired usage the program should print a usage with a proper error message, then finish.
- Exact usage description and error message text in all above cases are for your decision.
- The num_threads argument is optional.

The comparative run mode:

1. Should run all MissionControl implementations in the given folder, on all the configurations in the simulation yaml config, and with the given algorithm.
2. Should use num_threads as described below.
3. Should create an output directory directly under the mission_control_folder provided, with the name “comparative_results_<time>”. For the <time> part use code that will generate a new number per time to avoid collision with existing files if any.
   (You can see an example for getting time value into a string in this code example).
4. In case the folder cannot be created, write a proper error to screen.
5. The results folder should contain:
   a. All output map files, with a unique name per each file (you should decide on the unique name pattern, but should make it easy to relate each file to the mission that created it).
   b. Error log(s) files.
   c. A single Simulation Result Output File - as a YAML file
6. There will be a new YAML output file named: Comparative Simulation Result Output File (YAML) with the format as follows:

Comparative Simulation Result Output File - YAML

```
comparative_report:
  composition_file: "simulation_compositions.yaml"
  mission_control_folder: "folder"
  generated_at_utc: "2026-05-30T23:31:10Z"

  results_summary: # sorted by number of agreeing managers, descending
    - same_results: ["manager1.so", "manager2.so", "manager5.so"]
      total_score: 495
      total_steps: 100
    - same_results: ["manager3.so", "manager6.so"]
      total_score: 502
      total_steps: 124
    - same_results: ["manager4.so"]
      total_score: 495
      total_steps: 101

  errors: ["manager7.so", "manager8.so"] # could not be loaded / run
```

[...]

7. In addition to the above, all runs, per each mission control, will generate their own Simulation Result Output File - YAML as in assignment 2. With the name of the mission control added to the file name. Files will be added to the same output folder.

The competitive run mode:

1. Should run the given MissionControl using all algorithms.
2. Should use num_threads as described below.
3. Should create an output directory directly under the algorithms_folder provided, with the name “competition_<time>”. For the <time> part use code that will generate a new number per time, similarly as described for the comparative output file.
4. In case the folder cannot be created, write a proper error to screen.
5. The results folder should contain:
    a. All output map files, with a unique name per each file (you should decide on the unique name pattern, but should make it easy to relate each file to the mission that created it).
    b. Error log(s) files.
    c. A single Simulation Result Output File - as a YAML file
6. There will be a new YAML output file named: Competitive Simulation Result Output File (YAML) with the format as follows:

Competitive Simulation Result Output File - YAML

```
competitive_report:
  composition_file: "simulation_compositions.yaml"
  mission_control: "mission_control_filename.so"
  generated_at_utc: "2026-05-30T23:31:10Z"

  results_summary: # sorted by score descending, then by steps ascending
    - algorithm: "algorithm1.so"
      total_score: 495
      total_steps: 100
    - algorithm: "algorithm3.so"
      total_score: 490
      total_steps: 97
    - algorithm: "algorithm4.so"
      total_score: 490
      total_steps: 113

  errors: ["algorithm2.so", "algorithm5.so"] # could not be loaded / run
```

[...]

7. In addition to the above, all runs, per each algorithm, will generate their own Simulation Result Output File - YAML as in assignment 2. With the name of the algorithm added to the file name. Files will be added to the same output folder.

Threading model

- The num_threads command line argument sets the number of threads for the simulation, as follows:
  - If the argument is missing or if num_threads provided = 1, the program will use a single thread (the main thread).
  - If num_threads provided >= 2, the program will interpret <num_threads> as the requested number of threads for running the actual simulation in addition to the main thread.
  - Above means that the total number of threads will never be 2.
- Note that the exact number of threads may be lower than requested in the command line, in case there is no way to properly utilize the required number of threads you should not open threads which cannot be utilized (i.e. have nothing to run).
- Note that it is totally OK that your main thread will be waiting for all other threads in a join call (or any other similar blocking wait).
- If you prefer to load all the required .so files ahead it might be a proper solution!
  In case you find a way to avoid loading algorithms / mission_control instances simultaneously but rather load them (once!) only when needed and unload if not being used anymore (but without loading them again!) – you can ask for a bonus for that.
- It is better not to lock if you can avoid locking. But if you need to lock, you should of course lock.
- In case a result table can be known in advance, you can create it ahead (there is no need for a “sparse matrix” for managing results).
- Creating an Algorithm instance / MissionControl instance from their factories should be cheap. Do not cache instances, just recreate them using the factories when needed (this is NOT the same as unloading and loading the same .so file, which should be avoided).

API and Abstract Base Classes (Old and New)

Link to the skeleton

To allow common implementation you need to use common abstract classes. The actual header files would be published for use *as-is* and may/shall be in use by any of the three projects: *Simulator*, *MissionControl* and *Algorithm*.
Note that interfaces are not changed from assignment 2.

---

common/MappingAlgorithmFactory.h (code will be provided, with proper includes)

```
using MappingAlgorithmFactory =
       std::function<std::unique_ptr<IMappingAlgorithm>(MappingAlgorithmDependencies)>;
```

---

common/MissionControlFactory.h (code will be provided, with proper includes)

```
using MissionControlFactory =
       std::function<std::unique_ptr<IMissionControl>(MissionControlDependencies)>;
```

Automatic Registration

To allow simple discovery of the factories, we need to define a common registration process so all loaded classes would register themselves automatically.

Note: we will explain the process in class and if required will go through it in homework meeting in zoom. It may seem complicated but it is not too much.

You should use the following header files (the exact header file would be published and you would have to use it as-is) for auto registration:

```
// common/MappingAlgorithmRegistration.h
struct MappingAlgorithmRegistration {
  MappingAlgorithmRegistration(MappingAlgorithmFactory);
};

#define REGISTER_MAPPING_ALGORITHM(class_name)                                    \
    [[maybe_unused]] ::common::MappingAlgorithmRegistration register_me_##class_name{ \
        [](::common::MappingAlgorithmDependencies dependencies)                   \
            -> std::unique_ptr<::common::IMappingAlgorithm> {                     \
            return std::make_unique<class_name>(std::move(dependencies));          \
        }}
```

```
// common/MissionControlRegistration.h
struct MissionControlRegistration {
  MissionControlRegistration(MissionControlFactory);
};

#define REGISTER_MISSION_CONTROL(class_name)                                    \
    [[maybe_unused]] ::common::MissionControlRegistration register_me_##class_name{ \
        [](::common::MissionControlDependencies dependencies)                   \
            -> std::unique_ptr<::common::IMissionControl> {                     \
            return std::make_unique<class_name>(std::move(dependencies));        \
        }}
```

You are free to implement the .cpp files for the above auto registration classes as you wish but **you are not allowed to change the header files**!

Your concrete implementations for MappingAlgorithmImpl and MissionControlImpl will have the following lines in their .cpp, in the global scope:

```
REGISTER_MISSION_CONTROL(<class_name>)  e.g.:
REGISTER_MISSION_CONTROL(MissionControlImpl_098765432_123456789)
```

And similarly:

```
REGISTER_MAPPING_ALGORITHM(<class_name>)  e.g.:
REGISTER_MAPPING_ALGORITHM(MappingAlgorithmImpl_098765432_123456789)
```

The registered classes are unaware of the actual implementation of the registration process, which happens in the Simulation project (and will be explained to you). Note that the easiest way to implement the registration is to make the Simulation class (or a Registrar) a Singleton.

Note: the registration headers are in use by both the Simulator project and the Algorithm / MissionControl projects. However, the .cpp files of the registration **should be part of the Simulator project only** and their implementation is *on you* (you will be presented with a proposed implementation).

Additional Output and Error Handling

The MissionControl shall create output files with verbose info (anything you think is of interest), iff -verbose appears on the command line.
The MissionControl and Algorithm project may create error logs.
There is no need for the Simulator to handle a crash scenario of MissionControl / Algorithm, however in any other case the Simulator shall not crash.

Bonus

There would be a class competition and a bonus would be given for the best algorithms.
Additional bonuses can be requested as in previous exercises.

Submission

You shall submit a zip named ex3_<student1_id>_<student2_id>.zip that contains:

- ***5 folders*** - as described above
- ***4 makefiles*** (or CMakes) - 3 for each of the projects, inside the folder of the project that it builds. Additional one at the root (directly in the zip without any folder), for building all 3 projects.
- ***students.txt*** - a text file that includes one line per submitter with name and id
- ***README.md*** - info and remarks about your implementation - directly in the zip without any folder

DO NOT SUBMIT THE FOLLOWING FILES

- binary files
- external libraries (you may only use standard C++ libraries or libraries which were explicitly approved in the course forum)

Additional Notes and Requirements

1. As in assignment 2, you are not allowed to use *new* and *delete* in your code.

2. As in assignment 2, you should prefer using unique_ptr over shared_ptr – use shared_ptr only if there is actual need for sharing and the lifetime is unknown.

3. Do not forget to unload .so files before the program ends, by using the dlclose command. Make sure not to call dlclose if there are objects related to the .so which are still alive.

4. Note: the minimal requirement for your algorithm are:
    a. Do not fly the drone into walls.
    b. Try to map all the relevant surroundings in the configured boundaries.
    c. Try to be efficient and exact.

5. [TBD]